

Pharmacy Search: a working *search engine*, not yet a shop

A Vietnamese typo-tolerant pharmacy catalogue – 1,894 crawled products, a 15,306-word lexicon, and a Telex/VNI-aware correction engine – measured, compared against the Reember platform, and costed out for production.

Repo `D:\fpt` · branch `main` · 13 commits Live pharmacy-search-giabach.fly.dev
Stack `FastAPI` · `SQLite` · `Vite/vanilla JS` · `Fly.io` (`sin`)

121/122 TESTS PASSING 92.6% HELD-OUT TOP-1	51 API ENDPOINTS 374ms SEARCH P95	1,894 PRODUCTS	15,306 LEXICON KEYWORDS
--	---	--------------------------------------	---

01 What you actually built

The headline feature isn't the shop. It's the corrector — and that part is genuinely good work.

Vietnamese search breaks every assumption an English spell-checker makes. A user hunting for `bàn chải đánh răng` may type it accented, unaccented (`ban chai danh rang`), in full Telex (`banf chair ddanhs rawng`), in VNI (`ban2 chai3`), or — most often — half-finished, with the tone keys dropped wherever they got bored. Meanwhile half the catalogue is Latin drug names where those same letters are real: `aspirin` must not lose its `s` to a tone-key stripper.

`corrector.py` solves this properly. It parses candidate tokens against a real Vietnamese syllable grammar — onset, nucleus, coda drawn from three closed sets — so it can prove `tranhs` is `tranh` plus a tone key (because `tr` is a legal onset and `nh` a legal coda) while leaving `aspirin` untouched (nothing in it can be a coda). Every spelling then collapses to one *skeleton*, and matching becomes a substring test.

Around that sit four more deliberate decisions worth naming, because they're the difference between a demo and a system:

- **Keyboard-adjacency edit distance.** A `c` → `v` substitution costs 0.5 instead of 1.0, because they're neighbouring keys. `thuov` lands closer to `thuoc` than an equally-long but physically implausible typo does.
- **Precision over recall, enforced in code.** A fuzzy guess may only auto-apply at ≤1 edit. Two edits gets offered, never assumed — because `panadoL` → `panactoL` is a different drug. In a pharmacy that has to be the user's call, and the constant `FUZZY_AUTO_MAX_DIST` makes it a policy rather than an accident.

- **Phrase-first resolution.** Two thirds of the lexicon is multi-word, so a greedy longest-match runs before per-token correction. That's what separates `chống nắng` from `chóng nắng` — context a per-token corrector throws away.
- **A human-in-the-loop learning gate.** Query and click logs are mined for (*failed query* → *clicked query*) pairs, but nothing enters the live index without a pharmacist approving it in `learning/review.py` or the admin UI.

The commerce layer around it — auth, cart, wishlist, checkout, orders, coupons, inventory, admin reports — is competent and conventional: 51 endpoints, PBKDF2 at 390k iterations, soft-deleted products so order history survives, stock re-checked at order time rather than trusted from when the item was added, and revenue reporting that deliberately excludes `pending_payment` orders because those payment methods aren't connected to anything.

The code comments are unusually honest — they say what's fake and why. That's a real asset, and rarer than it should be.

02 Against Reember

Reember is broader. This is deeper. They fail in opposite directions.

Reember is a Django + Next.js retail inventory platform with a computer-vision pipeline — YOLO models shipped as `.onnx` and `.pt`, barcode processing, four Django apps with 43 migrations between them, JWT in `httpOnly` cookies, deployed to AWS App Runner. On paper it's the more ambitious system.

But the GitHub API tells a second story: created `2026-05-14T11:23:46Z`, last pushed `11:25:39Z` — the entire repository was created and pushed inside two minutes, in a single push, and never touched again. Its four `tests.py` files are Django's auto-generated stubs containing `« Create your tests here. »` and zero test functions. That's a code dump, not a maintained project.

AXIS	PHARMACY SEARCH	REEMBER
Automated tests	122 (121 pass, 1 strict xfail)	0 real — Django stubs only
Quality measurement	Held-out eval harness, latency percentiles, zero-result rate	None
Containerised	Multi-stage Dockerfile, reproducible build	No Dockerfile
Deploy config in repo	<code>fly.toml</code> , volume mount, healthcheck	None committed
Live and reachable	Yes — <code>/api/health</code> returns 15,306 keywords	URL in README; unverified
Commit history	13 commits, incremental	1 push, 2 minutes
Novel domain IP	Telex/VNI syllable parser + correction engine	CV models (pre-trained, vendored)
Feedback loop	Log mining → approval gate → index rebuild	None
Data provenance	<code>price_is_estimated</code> , <code>brand_is_estimated</code> , <code>stock_is_estimated</code>	Not evident
Token storage	localStorage — XSS-readable	httpOnly cookie
Schema migrations	<code>CREATE TABLE IF NOT EXISTS</code> + ad-hoc <code>ALTER</code>	Django migrations, 43 of them
Admin tooling	Hand-built dashboard (6 views)	Django admin, free
SSR / SEO capability	Hash-router SPA — invisible to crawlers	Next.js
README	None, anywhere in the repo	Root + frontend
CI pipeline	None	None
Licence	None	None

YOURS HAS, REEMBER DOESN'T

- **A test suite that runs.** 122 tests across 13 files, 29 seconds, covering auth, cart, checkout, catalog, inventory, admin and the corrector itself.
- **Measured quality, not claimed quality.** The held-out evaluation deliberately excludes two rule groups from the index, then tests *only* variants those rules produce — so the score can't be tautological. That methodology is more rigorous than most production search teams manage.
- **A reproducible build.** The Dockerfile rebuilds the search index from the committed catalogue at image-build time, so a fresh machine is never missing a derived artifact.
- **An honest failure record.** The latency test is `xfail(strict=True)` with a written explanation — a known gap that the suite will flag the moment it's fixed, rather than a deleted test.
- **Real deployment.** Single-origin static mount means no CORS layer, one process, one container.
- **A domain nobody can copy quickly.** The Vietnamese input-method handling is months of understanding, not a library import.

REEMBER HAS, YOURS DOESN'T

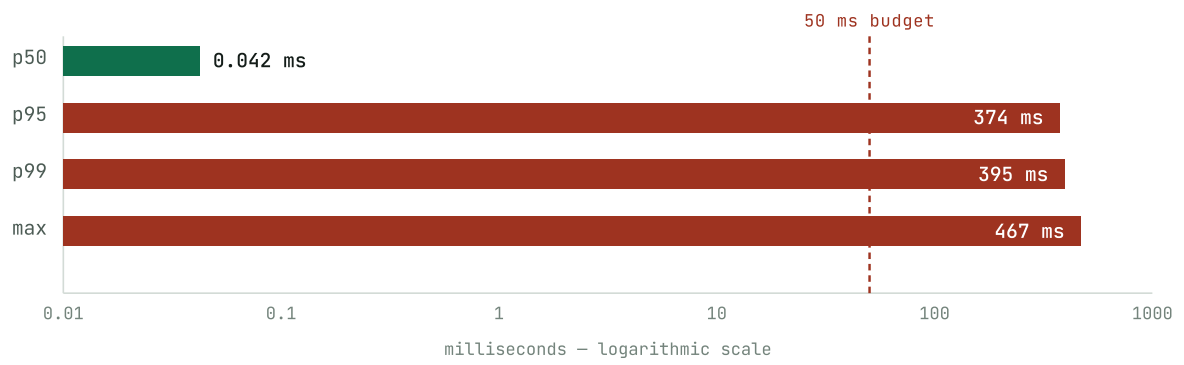
- **httpOnly cookie auth.** Genuinely more secure than your localStorage bearer token. Copy this.
- **A migration framework.** Your schema evolves by `IF NOT EXISTS` and a startup backfill loop; theirs has a versioned, reversible history.
- **Server-side rendering.** For a commercial shop, SEO isn't optional — and your hash router forecloses it entirely.
- **Free admin CRUD.** Django admin gave them for nothing what cost you six hand-written views.
- **A managed cloud platform.** AWS App Runner scales horizontally; your SQLite-on-a-volume design pins you to one machine.
- **An `.env.example`.** Small thing, but it's the difference between a repo someone can run and one they can't.

03 The numbers, measured

I ran the suite and the evaluation harness rather than reading the claims.

`pytest` : **121 passed, 1 xfailed** in 29.55s. The held-out evaluation — index built with the `pharma-suffix` and `inn-hdrop` rule groups removed, then tested on 500 variants generated *only* by those removed rules — scores **92.6% top-1, 93.6% top-3**, with **4.4%** producing no candidate at all. That means the engine recovers nine in ten typos it was never taught, through edit distance and the vocabulary views alone. It generalises.

Latency is where it falls over, and the shape of the failure is instructive.



Measured over 999 queries mixing exact matches, known typos and gibberish. The gap between p50 and p95 spans four orders of magnitude: a query that hits the indexed views returns in microseconds, while any token that reaches the fuzzy fallback scans the entire 15,306-word vocabulary and costs ~400 ms. This is not a slow system — it is a fast system with one uninstrumented path.

04 Findings, ranked

Fifteen issues, most severe first. Each carries the fix and the terms to search for.

F-01 Password reset returns the reset token in the HTTP response

CRITICAL

`POST /api/auth/forgot-password` returns `dev_only_reset_token` in its JSON body. Anyone who knows a registered email address can request that token and take over the account in two requests. The code comments this honestly and says to remove it before production — that removal has not happened, and the app is publicly reachable right now.

Fix: delete the token from the response body today. Send it by email instead, and until an email provider is wired up, disable the endpoint entirely rather than shipping it open. Also invalidate all existing sessions after a successful reset.

account takeover transactional email Resend / Postmark / SES
session invalidation on reset

F-02 Query logs and pharmacist approvals are written to ephemeral container storage

CRITICAL

The volume mounts at `/data` and holds only `app.db`. But `logs/queries.jsonl`, `logs/clicks.jsonl`, `learning/candidate_pairs.json` and `learning/approved_pairs.json` all resolve under `backend/` — inside the image. With `auto_stop_machines = 'stop'` and `min_machines_running = 0`, the machine stops whenever traffic idles. Every restart and every redeploy silently discards all search analytics *and* every correction a pharmacist approved. The learning loop — the most interesting thing in the project — cannot currently accumulate anything.

Fix: move both directories onto the volume via an env var (`PHARMACY_LOG_DIR=/data/logs`, and the learning JSON alongside it), matching how `PHARMACY_SEARCH_DB` already works. Better still, move approvals into a SQLite table so they're covered by the same backup as everything else, and emit logs as structured JSON to stdout for a log drain.

ephemeral filesystem Fly volumes structured logging stdout log drain 12-factor

F-03 Prescription-only medicines can be bought with no prescription check

CRITICAL

Products carry a `prescription` boolean and the crawled catalogue sets it — the very first row in `products.jsonl` is `"prescription": true`. Nothing in cart, checkout or order placement reads that flag. For a Vietnamese pharmacy this is not a feature gap; it is the regulatory core of the business, and it is the single thing that most separates "e-commerce demo" from "pharmacy".

Fix: block Rx items from the normal checkout path. Minimum viable version: flag them in the UI, require an uploaded prescription image plus a pharmacist review step before the order can move from `placed` to `shipped`. Model it as an order-level hold, not a cart-level block.

thuốc kê đơn

Rx verification workflow

pharmacist review queue

order hold state

Nghị định 117/2020

F-04 Session tokens live in localStorage

HIGH

Any successful XSS — one bad product name rendered as HTML, one compromised dependency — reads the bearer token and keeps it for the full 30-day TTL. Reember gets this right and you don't.

Fix: move to an `httpOnly; Secure; SameSite=Lax` cookie and add CSRF protection for state-changing requests. Shorten the session TTL and add a refresh path. Add a cleanup job — the `sessions` table currently never deletes expired rows.

httpOnly cookie

SameSite

CSRF double-submit

XSS

Content-Security-Policy

F-05 No rate limiting anywhere

HIGH

`/api/auth/login`, `/api/auth/register` and `/api/auth/forgot-password` accept unlimited attempts from one IP. PBKDF2 at 390k iterations is strong per-guess but also makes each login ~100 ms of CPU — so the login endpoint doubles as a cheap denial-of-service vector against a single shared machine.

Fix: add `slowapi` or a reverse-proxy limiter. Per-IP and per-account limits on auth routes, progressive backoff, and a lockout after repeated failures.

slowapi

credential stuffing

exponential backoff

account lockout

Cloudflare WAF

F-06 Concurrent checkouts can oversell stock

HIGH

`place_order` reads stock, validates it, then decrements with `UPDATE products SET stock = stock - ?`. Two orders interleaving between the read and the write both pass validation and both decrement — stock goes negative and you've sold inventory you don't have. Rare at current traffic; guaranteed at real traffic.

Fix: make the decrement itself the check — `UPDATE products SET stock = stock - ? WHERE id = ? AND stock ≥ ?` — and abort the transaction if `rowcount` is 0. Open the transaction with `BEGIN IMMEDIATE` so SQLite takes the write lock up front.

race condition

BEGIN IMMEDIATE

conditional UPDATE

optimistic concurrency

rowcount check

F-07 No database indexes beyond primary and unique keys

HIGH

Not one `CREATE INDEX` exists in the schema. Every cart read, order list, review lookup and session validation is a full table scan. It's invisible at 1,894 products and a handful of users, and it degrades linearly with every one you add.

Fix: index `cart_items(user_id)`, `orders(user_id, created_at)`, `order_items(order_id)`, `reviews(product_id)`, `wishlist_items(user_id)`, `sessions(expires_at)`. Separately: catalogue search uses `LIKE '%term%'`, which no B-tree index can serve — that one needs SQLite FTS5 over `search_text`.

`CREATE INDEX`

covering index

SQLite FTS5

`EXPLAIN QUERY PLAN`

leading-wildcard LIKE

F-08 Search p95 is 7.5× over its own stated budget

HIGH

Already documented as a strict `xfail`, which is the right way to carry a known gap — but it is still a 400 ms search on the path users hit precisely when they've made a typo, which is the feature's entire reason to exist.

Fix: stop scanning the full vocabulary. Build a character n-gram inverted index (or a BK-tree / SymSpell deletion index) to shortlist ~100 candidates before running edit distance. Cache the top few thousand queries.

SymSpell

BK-tree

character trigram index

candidate generation

LRU cache

F-09 No schema migration system

HIGH

Schema changes happen via `CREATE TABLE IF NOT EXISTS` plus an ad-hoc `ALTER TABLE` in `_ensure_search_text`. There is no version marker, no way to know what state a database is in, and no way to roll a change back. That function also rewrites *every* product row on every single boot — a full table scan at startup, as its own comment concedes.

Fix: adopt Alembic, or a minimal numbered-SQL-file runner with a `schema_version` table. Add a fold-version marker so the backfill only runs when the folding logic actually changed.

Alembic

`schema_version` table

forward-only migrations

idempotent startup

F-10 No security headers, no CI, no README, no licence

MEDIUM

No CSP, HSTS, `X-Content-Type-Options` or `X-Frame-Options`. No `.github/workflows`, so those 122 tests only run when you remember to run them. No README anywhere in the repo — nobody, including you in six months, can set this up without reading the Dockerfile. No licence, which legally means nobody may use it.

Fix: a `SecurityHeadersMiddleware`, one GitHub Actions workflow running pytest on every push and deploying on green, a README with setup and architecture, and a licence file. Half a day for all four; the largest credibility return in this list.

Content-Security-Policy

HSTS

GitHub Actions

branch protection

MIT / Apache-2.0

F-11 Product images hotlink Long Châu's CDN

MEDIUM

Every image loads from `cdn.nhathuoclongchau.com.vn`. They can rate-limit, hotlink-block or simply move the files, and your catalogue goes blank with no warning. It also leaks every visitor's IP to a competitor and sits on questionable legal ground for a commercial site.

Fix: mirror images to your own object storage behind a CDN, serve resized WebP/AVIF variants, and hold a licence or your own photography before charging anyone money.

asset mirroring

S3 / R2 + CDN

WebP / AVIF

image rights

lazy loading

F-12 The architecture is pinned to exactly one machine

MEDIUM

SQLite on a single Fly volume means you cannot run a second instance, so you cannot do a zero-downtime deploy and you have no redundancy. Compounding it, `min_machines_running = 0` means the first request after an idle period pays a cold start that includes parsing a 3.4 MB keyword file, a 6.9 MB near-miss file, the full `search_text` reconciliation scan, and a click-count aggregation.

Fix: for now, set `min_machines_running = 1` and move index loading behind a warmup. When you need to scale, migrate to Postgres — or keep SQLite and adopt LiteFS for replication.

cold start

lazy loading

LiteFS

Postgres migration

blue-green deploy

F-13 No backups

MEDIUM

A single Fly volume holds every user, order and review. Volumes are not backed up by default. One `fly volumes destroy`, one corrupted write, one region incident and the business is gone with no recovery path.

Fix: run Litestream as a sidecar, streaming SQLite's WAL to S3 or Cloudflare R2 continuously. Then actually rehearse a restore — an untested backup is a guess.

Litestream

WAL streaming

point-in-time recovery

restore drill

RPO / RTO

F-14 A hash-router SPA is invisible to search engines

MEDIUM

Routes are `#/product/123`, rendered entirely client-side. Google will not index individual products, and no product page can be shared with a preview. For an e-commerce site where organic search is the main acquisition channel, this is a commercial defect, not a technical one.

Fix: switch to History API paths with a server-side SPA fallback, add per-product `<title> /meta/Open Graph tags` and `Product` JSON-LD, and generate a sitemap. If organic traffic matters seriously, server-render the product pages.

History API routing

JSON-LD Product schema

Open Graph

sitemap.xml

SSR / prerender

F-15 Interface language and catalogue language don't match

MEDIUM

The chrome is English — "Typo-tolerant product search", "Cash on delivery", "Standard (3-5 days)" — while every product name, and the entire premise of the correction engine, is Vietnamese. The users who need Telex-tolerant search are the users reading Vietnamese.

Fix: extract UI strings into a locale file and ship Vietnamese first. Also localise money properly — prices are integer VND and should render as 145.000 đ, not with English separators.

i18n string extraction

vi-VN locale

Intl.NumberFormat

currency formatting

05 Deploying it properly

You already deploy. What's missing is everything around the deploy that makes it repeatable, reversible and observable.

These steps are ordered — each depends on the one before it. Steps 1 through 4 are the difference between "it's on the internet" and "it's in production".

01 Separate the environments

One app that is both your test bed and your live site means every experiment is a production incident. Run two Fly apps from the same repo with separate volumes and secrets.

```
# staging gets its own app, volume and database
fly apps create pharmacy-search-staging
fly volumes create pharmacy_data --app pharmacy-search-staging --region sin --size 1
fly deploy --app pharmacy-search-staging --config fly.staging.toml
```

02 Move every secret out of the image

Nothing sensitive belongs in `fly.toml` or the Dockerfile — those are committed. Use Fly secrets, which are injected as environment variables at runtime and never appear in an image layer.

```
fly secrets set SESSION_SIGNING_KEY=$(openssl rand -hex 32) \
    EMAIL_API_KEY=... \
    SENTRY_DSN=...
```

Commit a `.env.example` listing every variable with dummy values, so the repo documents its own configuration.

03 Make the tests a gate, not a habit

One workflow: run pytest on every push and pull request; deploy to staging on green merges to main; deploy to production only on a version tag. Add branch protection so main cannot be pushed to directly.

```
# .github/workflows/ci.yml - the shape
on: [push, pull_request]
jobs:
  test:    pytest backend/tests
  build:   docker build .
  deploy:  flyctl deploy --remote-only # needs FLY_API_TOKEN secret
```

04 Back up continuously, and rehearse the restore

Litestream replicates SQLite's write-ahead log to object storage as it happens, giving point-in-time recovery for a few cents a month. Run it as the container's supervisor process alongside uvicorn.

- Replicate to S3 or Cloudflare R2 from within the container.
- Snapshot the volume on a schedule as a second, independent line of defence.
- **Restore into staging once a quarter.** A backup you have never restored is not a backup.

05 Wire up health checks and alerting

`/api/health` already exists and already reports lexicon size — good instinct. Register it with Fly so a failed deploy rolls back instead of serving errors.

```
[[http_service.checks]]
  interval = "15s"
  timeout  = "3s"
  method   = "GET"
  path     = "/api/health"
```

Add Sentry for exceptions and an external uptime monitor that pages you when the site is down — because the machine that would have told you is the one that's down.

06 Put a CDN and a real domain in front

- `fly certs add nhathuoc.example.vn` for TLS on your own domain.
- Cloudflare in front for CDN caching of static assets, DDoS absorption and a WAF.
- Cache `/assets/*` hard (Vite already content-hashes filenames); never cache `/api/*`.

07 Know how to roll back before you need to

Every deploy is a release you can return to. Practise this once while nothing is wrong.

```
fly releases --app pharmacy-search-giabach
fly deploy --image registry.fly.io/pharmacy-search-giabach:<previous>
```

The caveat: a rollback restores *code*, not schema. Once you have migrations (F-09), write them so the previous release can still read the new schema — additive changes first, destructive ones a release later.

06 Running it, week to week

WEEKLY

- Review the zero-result rate from `eval/query_log.py` — every zero-result query is a product you don't stock or a word the lexicon is missing.
- Work the pharmacist approval queue in the admin corrections view; rebuild the index after approving.
- Check low-stock counts from the admin summary.
- Skim Sentry for new exception signatures.

MONTHLY

- Re-crawl the catalogue; diff prices and stock rather than replacing rows wholesale, so the estimated-value flags stay meaningful.
- Re-run the full evaluation harness and record the numbers — you want a time series, not a single reading.
- `pip list --outdated` and `npm audit`; patch, don't accumulate.
- Review latency percentiles against the 50 ms budget.

QUARTERLY

- Restore a backup into staging end to end.
- Rotate secrets and API keys.
- Re-read the honest comments in the codebase and check which caveats are now false — `dev_only_reset_token` is the first.
- Review the growth curve: when a single machine stops being enough, the Postgres migration should be planned, not panicked.

COLLECT WHAT YOU CAN'T MEASURE YET

- **Correction precision** — metric #1 in your own harness — still can't be computed, because the query log doesn't link consecutive searches into a session with an outcome. Add a `session_id` + outcome field and it becomes measurable.
- **The human-typed test set** is an empty TSV with only a header. Sit ten people down, have them type 100 product names quickly, and record it. Generated typos have a different error distribution from real ones, and until this file has rows, the 92.6% is a lower-bound proxy, not a user-facing accuracy.

07 The road to commercial

Four phases. Don't reorder them — each one is worthless without the one before.

PHASE 1 • THIS WEEK

Stop the bleeding

- Remove `dev_only_reset_token` (F-01)
- Move logs and approvals onto the volume (F-02)
- Add rate limiting on auth routes (F-05)
- Fix the stock race (F-06)
- Add database indexes (F-07)
- Write a README and pick a licence

PHASE 2 • THIS MONTH

Make it operable

- CI pipeline with tests as a gate (F-10)
- Litestream backups + a restore drill (F-13)
- Sentry, health checks, structured logs
- Staging environment (step 01)
- Migration framework (F-09)
- Security headers and cookie auth (F-04)

PHASE 3 • THIS QUARTER

Make it a pharmacy

- Prescription verification workflow (F-03)
- Real payments — VNPay or Momo, never hand-rolled
- Transactional email: receipts, shipping, resets
- Vietnamese-first UI and VND formatting (F-15)
- Self-hosted product images (F-11)
- Terms, privacy policy, PDPA compliance

PHASE 4 • WHEN IT GROWS

Make it fast and findable

- N-gram candidate index for the fuzzy path (F-08)
- FTS5 for catalogue search (F-07)
- History-API routing, JSON-LD, sitemap (F-14)
- Postgres or LiteFS for multi-machine (F-12)
- Order cancellation, refunds, returns
- Accessibility audit against WCAG 2.2 AA

- The honest verdict

You have built one genuinely impressive thing and wrapped it in a competent but ordinary shop. The Vietnamese correction engine — the syllable grammar, the keyboard-weighted distance, the refusal to auto-correct a drug name at two edits — is the kind of work that gets you hired. The held-out evaluation methodology is more rigorous than most teams bother with. Keep leading with that, in your README and in interviews; nobody will be impressed by another cart.

Compared to Reember, the gap isn't ambition, it's follow-through. Reember has more surface area and a machine-learning story, but it was pushed once and abandoned with zero tests. Yours has 122 tests, an evaluation harness, a Dockerfile, and a URL that answers. In engineering terms that comparison isn't close — and the two things Reember does better, httpOnly cookies and migrations, are both a day's work to adopt.

What stands between this and commercial is not features. It's F-01 through F-03: a live account-takeover vector, a learning loop that silently discards its own data, and prescription medicines sold with no prescription. Fix those three and you have a defensible product. Everything after them is refinement.

Prepared 3 September 2026 from a read of the working tree at commit `3eb5346`. Test results, evaluation scores and latency percentiles were produced by running `pytest backend/tests` and `python eval/evaluate.py` locally, not taken from documentation. Reember figures come from the GitHub REST API and public file tree. The live health check was queried at `pharmacy-search-giabach.fly.dev/api/health`.